CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 9

Secure User Registration with Database Insert, Duplicate Email Check and Password Hashing

PHP 8 · MySQLi prepared statements and transactions · password_hash() (bcrypt) · CSRF tokens · studenthub_db

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 9 report (`Practical-9/PRAC_9_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report describes a standalone registration page in `Practical-9/` with its own database `studenthub_prac9` (tables `users` and `audit_logs`). This corrected version describes registration as it now works in the integrated portal (`studenthub/register.php`, database `studenthub_db`): the same security requirements, but one transaction now creates the login account **and** its linked student record with the next enrollment number, and the new student is signed in through the portal's one authentication system. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To build a secure StudentHub registration page in PHP that validates the input, refuses duplicate e-mail addresses and usernames, stores the password only as a hash created with `password_hash()` and inserts the new user into MySQL with MySQLi prepared statements – inside the integrated portal, so that a new student immediately has a working account, a student record and an enrollment number.

- Validate every field on the server (the browser can be bypassed); browser checks are only a convenience.
- Refuse an e-mail address or username that is already registered – in PHP and with UNIQUE keys in MySQL.
- Store passwords only as `password_hash()` hashes; never show, log or save a plaintext password.
- Use MySQLi prepared statements for every query with user input, and one transaction for all inserts.
- Extensions: CSRF token validation and an audit-log entry for every successful registration.
- Integration: create the `students` record linked through `students.user_id`, assign the next enrollment number and sign the student in; the role is always `student`, never taken from the form.

2. Problem Definition

Official title: *"Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing."* A registration form is the first part of a portal an attacker can reach. Without server-side checks it accepts invalid data; without duplicate checks one e-mail address can own several accounts; plaintext passwords are exposed by any database leak; and SQL built from user input allows SQL injection.

In the portal one more problem had to be solved: an account alone is not enough. Profile, dashboard and event registration need a `students` record that belongs to the account. Before the integration, accounts without a record showed a fixed name and roll number (TEST_REPORT §2). Registration therefore has to create both rows together – or neither – and must not take over a record that an administrator created for someone else.

#	Evaluation requirement	Where it is shown
1	Successful user registration	Section 10, Figures 2–3; test "valid registration -> 303 to dashboard.php"
2	Empty-field validation	Section 7 (a "required" message for every field)
3	Invalid email validation	Section 7; test "invalid email -> rejected"
4	Password mismatch validation	Section 7; test "passwords do not match -> rejected"
5	Duplicate email / username validation	Section 8; tests for both, also in another letter case
6	Password stored as a hash	Section 9; test "password stored only as a bcrypt hash that password_verify() accepts"
7	Successful database insertion	Section 10, Figure 5; test "account + student record created together"

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Validation, sessions, CSRF tokens, `password_hash()`; PHP's built-in server for the tests
MySQLi (exception mode)	Prepared statements and the registration transaction
MariaDB 10.4.32 (XAMPP)	Shared portal database `studenthub_db` – tables `users`, `students`, `audit_logs`
HTML5, CSS3, JavaScript	Form (`register.php`, `assets/css/auth.css`); browser checks `assets/js/register-validation.js` (added with Practical 5)
Python 3 (standard library)	`tests/test_auth.py` – HTTP tests on a disposable database
Microsoft Edge (headless) + Playwright	`tests/browser_auth.py`, `tests/browser_e2e.py` – browser checks and screenshots
Git / GitHub	Commit `8011d39` – Practicals 9 and 10 integrated into the portal

4. Place in the Integrated Portal

Registration is the public page `studenthub/register.php` (linked from the sign-in page as *Create an account*). It uses the portal's shared connection, authentication and audit log:

File	Role in Practical 9
`register.php`	Form and controller: CSRF check, validation, registration, sign-in, audit entry, redirect
`includes/registration.php`	`validate_registration()`, `find_registration_conflicts()`, `next_enrollment_number()`, `register_student_account()`
`includes/auth.php`	`password_problems()` and `PASSWORD_POLICY_TEXT` (shared with the password change), `csrf_field()` / `csrf_is_valid()`, `login_user()`, `e()`
`audit_logger.php`, `db.php`	`log_audit()` (USER_REGISTER); the MySQLi connection `$conn`
`assets/js/register-validation.js`	Real-time browser checks with the same rules (later Practical 5 integration)

5. Database Design

Registration writes to two tables of `studenthub_db` (InnoDB, `utf8mb4_unicode_ci`). Their current structure in `database/studenthub_demo.sql`:

database/studenthub_demo.sql (lines 39–74) – users and students (current schema)

```
39  CREATE TABLE IF NOT EXISTS `users` (
40    `id` int(11) NOT NULL AUTO_INCREMENT,
41    `username` varchar(50) NOT NULL,
42    `email` varchar(100) NOT NULL,
43    `password` varchar(255) NOT NULL,
44    `role` varchar(20) NOT NULL DEFAULT 'student',
45    `status` enum('active','disabled') NOT NULL DEFAULT 'active',
46    `created_at` timestamp NULL DEFAULT current_timestamp(),
47    `last_login` datetime DEFAULT NULL,
48    `remember_token_hash` varchar(255) DEFAULT NULL,
49    `remember_token_expiry` datetime DEFAULT NULL,
50    PRIMARY KEY (`id`),
51    UNIQUE KEY `username` (`username`),
52    UNIQUE KEY `email` (`email`),
53    CONSTRAINT `chk_users_role` CHECK (`role` in ('admin','editor','student'))
54  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
55
```

```
56  -- students.user_id: optional link to the login account (one account per student record)
57  -- students.enrollment_no: unique (migration 002); NULL is allowed for legacy rows
58  CREATE TABLE IF NOT EXISTS `students` (
59    `id` int(11) NOT NULL AUTO_INCREMENT,
60    `user_id` int(11) DEFAULT NULL,
61    `enrollment_no` varchar(20) DEFAULT NULL,
62    `name` varchar(100) NOT NULL,
63    `email` varchar(100) NOT NULL,
64    `course` varchar(50) NOT NULL,
65    `gender` varchar(10) NOT NULL,
66    `semester` int(11) DEFAULT 3,
67    `status` varchar(20) DEFAULT 'Active',
68    `created_at` timestamp NULL DEFAULT current_timestamp(),
69    PRIMARY KEY (`id`),
70    UNIQUE KEY `email` (`email`),
71    UNIQUE KEY `uq_students_user_id` (`user_id`),
72    UNIQUE KEY `uq_students_enrollment_no` (`enrollment_no`),
73    CONSTRAINT `fk_students_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE SET NULL
    ON UPDATE CASCADE,
74    CONSTRAINT `chk_students_semester` CHECK (`semester` is null or `semester` between 1 and 8)
```

- `users.password` holds only the `password_hash()` result (VARCHAR(255) leaves room for longer future algorithms). `username` and `email` are UNIQUE; because the collation ignores letter case, `Diya_Mehta` counts as a duplicate of `diya_mehta`. `status` and `chk_users_role` were added later by migration 004 (Practical 14); the columns `remember_token_hash` / `remember_token_expiry` are legacy columns that the portal does not use.
- `students.user_id` (migration 001, Practical 8) links the record to its account: UNIQUE (one account per record) with `ON DELETE SET NULL`. `students.email` is UNIQUE, `enrollment_no` UNIQUE since migration 002, and `semester` is checked to be NULL or 1–8.
- Registration inserts the role as the literal `'student'` and the status as `'Active'`; neither value can come from the form.

6. Registration Form and Browser-side Checks

`register.php` shows one card with these inputs; each has a visible label, a suitable HTML type, length limits and `autocomplete` hints. A hidden field carries the session's CSRF token. The form has `novalidate`, so the server alone decides; a field with an error gets `aria-invalid="true"` and `aria-describedby` pointing to its message.

Field	Input	Server rule (validate_registration)
Full Name (`name`)	text, 2–100	letters (any language), spaces, apostrophes, dots, hyphens; extra spaces collapsed
Username	text, 3–30, pattern	starts with a letter; then letters, digits, underscore
Email Address	email, max 100	`FILTER_VALIDATE_EMAIL`; trimmed and stored in lower case
Gender / Course / Admission Year	radio / select / select	Male, Female or Other; one of 4 courses; the current academic year or one of the two before
Password / Confirm Password	password, 8–72	policy of `password_problems()`; confirmation equal (`hash_equals()`)
Terms (`agree`)	checkbox value 1	must be exactly `'1'`

With the later Practical 5 integration, `assets/js/register-validation.js` checks every field while typing with the same rules and messages, shows a password-strength meter and keeps the form from being sent while a rule is broken (Figure 1). This is only a convenience: `tests/test_forms.py` confirmed that a request bypassing the browser is still refused by the server ("the server refuses the weak password (422, nothing stored)").

7. Server-side Validation

`validate_registration()` first makes sure every value is a plain UTF-8 string – a request such as `name[]=x` sends an array and gets "This field contains characters that are not allowed." – and then applies the rules. Every field has its own "required" message (for example "Please enter your full name.", "Please choose a username."), which covers the empty-field requirement. The password policy is shared with the password change in the profile:

studenthub/includes/auth.php (lines 293–317) – password policy: 8–72, upper, lower, digit, special character

```
293  function password_problems(string $password): array
294  {
295      $problems = [];
296      if (mb_strlen($password, 'UTF-8') < 8) {
297          $problems[] = 'at least 8 characters';
298      }
299      if (strlen($password) > 72) {                       // bcrypt uses only the first 72 bytes
300          $problems[] = 'at most 72 characters';
301      }
302      if (!preg_match('/[A-Z]/', $password)) {
303          $problems[] = 'an uppercase letter';
304      }
305      if (!preg_match('/[a-z]/', $password)) {
306          $problems[] = 'a lowercase letter';
307      }
308      if (!preg_match('/[0-9]/', $password)) {
309          $problems[] = 'a number';
310      }
311      if (!preg_match('/[^A-Za-z0-9]/', $password)) {
312          $problems[] = 'a special character such as ! @ # ?';
313      }
314      return $problems;
315  }
316
317  const PASSWORD_POLICY_TEXT = 'At least 8 characters (max. 72) with an uppercase letter, a lowercase
     letter, a number and a special character.';
```

studenthub/includes/registration.php (lines 78–115) – username, e-mail, lists, password, confirmation and terms

```
 78      if (!isset($errors['username'])) {
 79          if ($username === '') {
 80              $errors['username'] = 'Please choose a username.';
 81          } elseif (strlen($username) < 3 || strlen($username) > 30) {
 82              $errors['username'] = 'The username must be 3–30 characters long.';
 83          } elseif (!preg_match('/^[A-Za-z][A-Za-z0-9_]*$/', $username)) {
 84              $errors['username'] = 'Start with a letter; then use letters, numbers and underscores
     only.';
 85          }
 86      }
 87      if (!isset($errors['email'])) {
 88          if ($email === '') {
 89              $errors['email'] = 'Please enter your email address.';
 90          } elseif (strlen($email) > 100 || filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
 91              $errors['email'] = 'Please enter a valid email address, e.g. name@example.com.';
 92          }
 93      }
 94      if (!isset($errors['gender']) && !in_array($raw['gender'], REGISTRATION_GENDERS, true)) {
 95          $errors['gender'] = 'Please choose Male, Female or Other.';
 96      }
 97      if (!isset($errors['course']) && !isset(REGISTRATION_COURSES[$raw['course']])) {
 98          $errors['course'] = 'Please select a course from the list.';
 99      }
100      if (!isset($errors['adm_year']) && !in_array($raw['adm_year'], registration_admission_years(),
     true)) {
101          $errors['adm_year'] = 'Please select an admission year from the list.';
102      }
103      if (!isset($errors['password'])) {
104          $problems = password_problems($raw['password']);
105          if ($raw['password'] === '') {
106              $errors['password'] = 'Please choose a password.';
107          } elseif ($problems) {
108              $errors['password'] = 'The password needs ' . implode(', ', $problems) . '.';
109          }
110      }
```

```
111      if (!isset($errors['confirm_password']) && !isset($errors['password']) &&
     !hash_equals($raw['password'], $raw['confirm_password'])) {
112          $errors['confirm_password'] = 'The passwords do not match.';
113      }
114      if (($input['agree'] ?? null) !== '1') {
115          $errors['agree'] = 'Please accept the Terms & Conditions.';
```

The maximum of 72 is measured in bytes because bcrypt uses only the first 72 bytes. The semester is not a form field: it is derived from the admission year (admitted this academic year → 1, last year → 3, two years ago → 5). Extra fields such as `role=admin` are ignored. When there are errors the page answers HTTP 422 with "Please correct the highlighted fields.", a message under each field and the entered values (never the passwords) shown again through `htmlspecialchars()`.

8. Duplicate Email and Username Protection

After validation, `find_registration_conflicts()` runs separate prepared statements, so the user is told exactly which value is taken. The e-mail is compared trimmed and in lower case. If no account but a **student record** already has the address (for example one created by an administrator), the registration is refused with a request to contact the administrator – the record is never linked or changed automatically.

studenthub/includes/registration.php (lines 130–159) – username, account e-mail and student-record e-mail checks

```
130  function find_registration_conflicts(mysqli $db, string $username, string $email): array
131  {
132      $errors = [];
133      $stmt = $db->prepare('SELECT 1 FROM users WHERE username = ? LIMIT 1');
134      $stmt->bind_param('s', $username);
135      $stmt->execute();
136      if ($stmt->get_result()->num_rows > 0) {
137          $errors['username'] = 'This username is already taken. Please choose another one.';
138      }
139      $stmt->close();
140
141      $stmt = $db->prepare('SELECT 1 FROM users WHERE LOWER(TRIM(email)) = ? LIMIT 1');
142      $stmt->bind_param('s', $email);
143      $stmt->execute();
144      if ($stmt->get_result()->num_rows > 0) {
145          $errors['email'] = 'An account with this email address already exists. Please sign in
     instead.';
146      }
147      $stmt->close();
148
149      if (!isset($errors['email'])) {
150          $stmt = $db->prepare('SELECT 1 FROM students WHERE LOWER(TRIM(email)) = ? LIMIT 1');
151          $stmt->bind_param('s', $email);
152          $stmt->execute();
153          if ($stmt->get_result()->num_rows > 0) {
154              // A record created by the administrator (or an older record): do not link it
     automatically
155              $errors['email'] = 'A student record with this email address already exists. Please
     contact the administrator to activate your account.';
156          }
157          $stmt->close();
158      }
159      return $errors;
```

A check in PHP alone is not enough: two requests can both pass it before either inserts. The UNIQUE keys on `users.username`, `users.email` and `students.email` have the final word – an INSERT that fails with MySQL error 1062 rolls the transaction back and returns the same friendly message (Section 10).

9. Password Hashing

A hash is a one-way function: the password can be turned into the hash, but not back. `password_hash($password, PASSWORD_DEFAULT)` uses bcrypt (`$2y$…`) in PHP 8.2 and includes a random salt, so the same password gives a different hash every time and precomputed tables are useless. At sign-in

(Practical 10) `password_verify()` recomputes the hash with the stored salt and cost. Only the hash is stored; the error log line of a failed registration contains no password or hash, and no password appears in this report or the screenshots (the fields are masked). The tests confirmed that the stored value is a bcrypt hash accepted by `password_verify()` and that no password appears in the server log.

10. One Transaction: Account, Student Record and Enrollment Number

Enrollment numbers follow the portal's existing scheme: the last two digits of the admission year, the course code (IT, CS, DS or WD) and a three-digit counter, e.g. `26IT001`. The next number is computed while the matching rows are locked with `SELECT … FOR UPDATE`, so two simultaneous registrations never get the same number:

studenthub/includes/registration.php (lines 163–179) – next free number for the prefix

```
163  function next_enrollment_number(mysqli $db, string $prefix): string
164  {
165      $like = $prefix . '%';
166      $stmt = $db->prepare('SELECT enrollment_no FROM students WHERE enrollment_no LIKE ? FOR UPDATE');
167      $stmt->bind_param('s', $like);
168      $stmt->execute();
169      $max = 0;
170      foreach ($stmt->get_result()->fetch_all(MYSQLI_NUM) as [$number]) {
171          if (preg_match('/^' . preg_quote($prefix, '/') . '(\d{3})$/', (string) $number, $m)) {
172              $max = max($max, (int) $m[1]);
173          }
174      }
175      $stmt->close();
176      if ($max >= 999) {
177          throw new RuntimeException("No enrollment numbers left for prefix $prefix.");
178      }
179      return $prefix . sprintf('%03d', $max + 1);
```

`register_student_account()` checks the conflicts, hashes the password and then creates both rows in **one transaction**: the account (role always `'student'`) and the student record with `user_id` = the new account's id. On any error the transaction is rolled back, so an account never exists without its record and vice versa.

studenthub/includes/registration.php (lines 182–225) – account + linked record, commit or rollback

```
182  // Creates the account and the linked student record in one transaction.
183  // Returns ['user_id', 'student_id', 'enrollment_no'] or ['errors' => [...]]; throws RuntimeException
     on server problems.
184  function register_student_account(mysqli $db, array $clean): array
185  {
186      try {
187          $conflicts = find_registration_conflicts($db, $clean['username'], $clean['email']);
188          if ($conflicts) {
189              return ['errors' => $conflicts];
190          }
191          $hash   = password_hash($clean['password'], PASSWORD_DEFAULT);
192          $prefix = substr($clean['adm_year'], 2, 2) . REGISTRATION_COURSES[$clean['course']];
193
194          $db->begin_transaction();
195          try {
196              $enrollment = next_enrollment_number($db, $prefix);
197
198              $stmt = $db->prepare("INSERT INTO users (username, email, password, role) VALUES (?, ?,
     ?, 'student')");
199              $stmt->bind_param('sss', $clean['username'], $clean['email'], $hash);
200              $stmt->execute();
201              $user_id = (int) $stmt->insert_id;
202              $stmt->close();
203
204              $stmt = $db->prepare("INSERT INTO students (user_id, enrollment_no, name, email, course,
     gender, semester, status)
205                                    VALUES (?, ?, ?, ?, ?, ?, ?, 'Active')");
206              $stmt->bind_param('issssssi', $user_id, $enrollment, $clean['name'], $clean['email'],
     $clean['course'], $clean['gender'], $clean['semester']);
207              $stmt->execute();
208              $student_id = (int) $stmt->insert_id;
209              $stmt->close();
```

```
210
211             $db->commit();
212             return ['user_id' => $user_id, 'student_id' => $student_id, 'enrollment_no' =>
    $enrollment];
213         } catch (Throwable $e) {
214             $db->rollback();
215             if ($e instanceof mysqli_sql_exception && $e->getCode() === 1062) {      // registered by
    someone else meanwhile
216                 return ['errors' => str_contains($e->getMessage(), "'username'")
217                     ? ['username' => 'This username is already taken. Please choose another one.']
218                     : ['email' => 'This email address is already registered. Please sign in or
    contact the administrator.']];
219             }
220             throw $e;
221         }
222     } catch (Throwable $e) {
223         // Logged without any password or hash
224         error_log('StudentHub registration failed: ' . get_class($e) . ' [' . $e->getCode() . '] ' .
    $e->getMessage());
225         throw new RuntimeException('Registration could not be completed.');
```

The rollback was tested by a test trigger that makes the student INSERT fail after the account INSERT: the page showed a general message without technical details, the details went to the server log, and neither the account nor the record existed afterwards.

11. CSRF Protection, Sign-in and Audit Log

The controller at the top of `register.php` shows the order of the steps. A POST is processed only if it carries the token of the session (`csrf_is_valid()` compares with `hash_equals()`); otherwise nothing is validated or saved. After a successful registration `login_user()` signs the student in – it regenerates the session ID and creates a new CSRF token (Practical 10) – the audit entry USER_REGISTER is written, and a 303 redirect leads to the dashboard (Post/Redirect/Get), where a one-time message names the enrollment number.

studenthub/register.php (lines 15–43) – controller: CSRF, validation, registration, sign-in, audit entry

```
15  if ($_SERVER["REQUEST_METHOD"] === "POST") {
16      if (!csrf_is_valid($_POST["csrf_token"] ?? null)) {
17          $error = "The registration form expired. Please check your details and submit again.";
18      } else {
19          [$clean, $errors] = validate_registration($_POST);
20          foreach (array_keys($values) as $key) {
21              $values[$key] = $clean[$key] ?? $values[$key];
22          }
23          if (!$errors) {
24              try {
25                  $result = register_student_account($conn, $clean);
26                  if (isset($result["errors"])) {
27                      $errors = $result["errors"];
28                  } else {
29                      login_user(["id" => $result["user_id"], "username" => $clean["username"], "email"
    => $clean["email"], "role" => "student"]);
30                      log_audit($conn, $result["user_id"], $clean["username"], "USER_REGISTER",
31                          "New student account and linked student record created (enrollment " .
    $result["enrollment_no"] . ").", "SUCCESS");
32                      flash("success", "Welcome to StudentHub, " . $clean["name"] . "! Your enrollment
    number is " . $result["enrollment_no"] . ".");
33                      redirect("dashboard.php");
34                  }
35              } catch (RuntimeException $e) {
36                  $error = "Your account could not be created because of a server problem. Nothing was
    saved. Please try again later.";
37              }
38          }
39      }
40      if ($error === "" && $errors) {
41          $error = "Please correct the highlighted fields.";
42      }
43      http_response_code(422);
```

A visitor who is already signed in is redirected away from the page. The CSRF token is 32 random bytes from `random_bytes()` (64 hexadecimal characters) per session; a missing or wrong token gives the message "The registration form expired. Please check your details and submit again." and stores nothing.

12. Output Screenshots

Genuine full-page captures from Microsoft Edge (headless, Playwright, window 1280 px wide) of the integrated portal on PHP's built-in server with a disposable database built from `database/studenthub_demo.sql`; all names and addresses are fictional and every password field is masked. Figures 1, 3 and 5 come from `tests/browser_auth.py`, Figures 2 and 4 from the end-to-end test `tests/browser_e2e.py`.

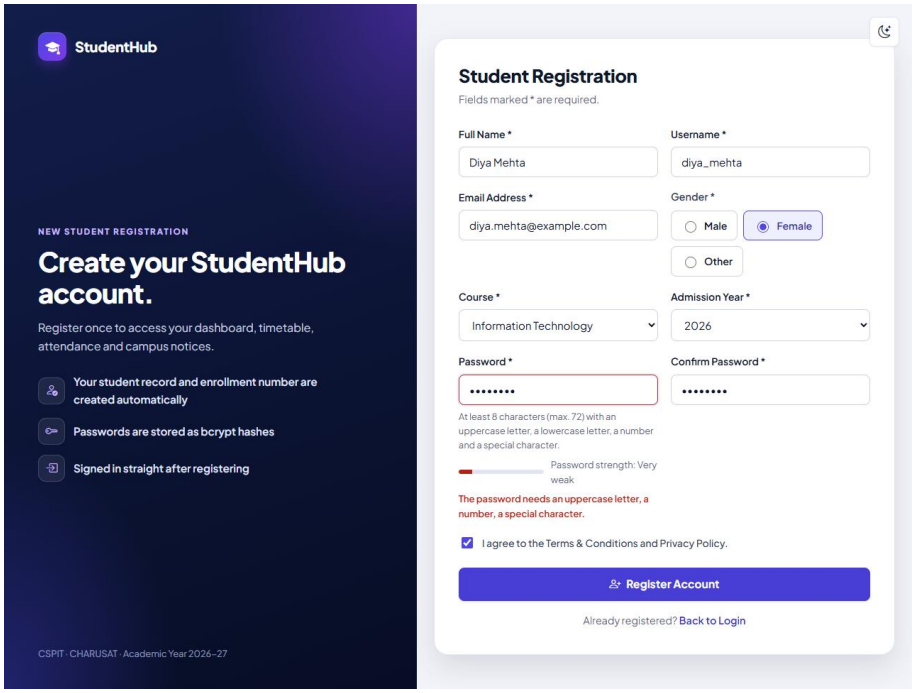


Figure 1: Weak password "weakpass" (masked) for Diya Mehta: the browser-side check added later with the Practical 5 integration stops the form – strength "Very weak" and "The password needs an uppercase letter, a number, a special character."; nothing was sent and the values stay in the fields. The server-side refusal of a weak password is tested over HTTP in tests/test_auth.py (register-weak-password.png)

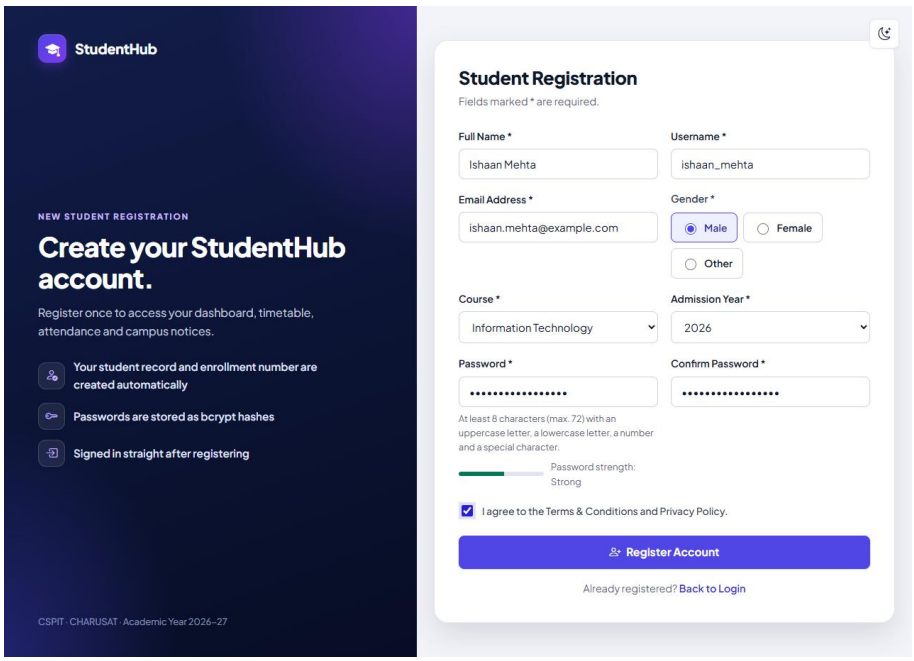


Figure 2: The registration form filled for the fictional student Ishaan Mehta (ishaan_mehta, ishaan.mehta@example.com, Information Technology, admission year 2026) with a strong password (strength "Strong") and the terms accepted, just before Register Account (01-registration.png)

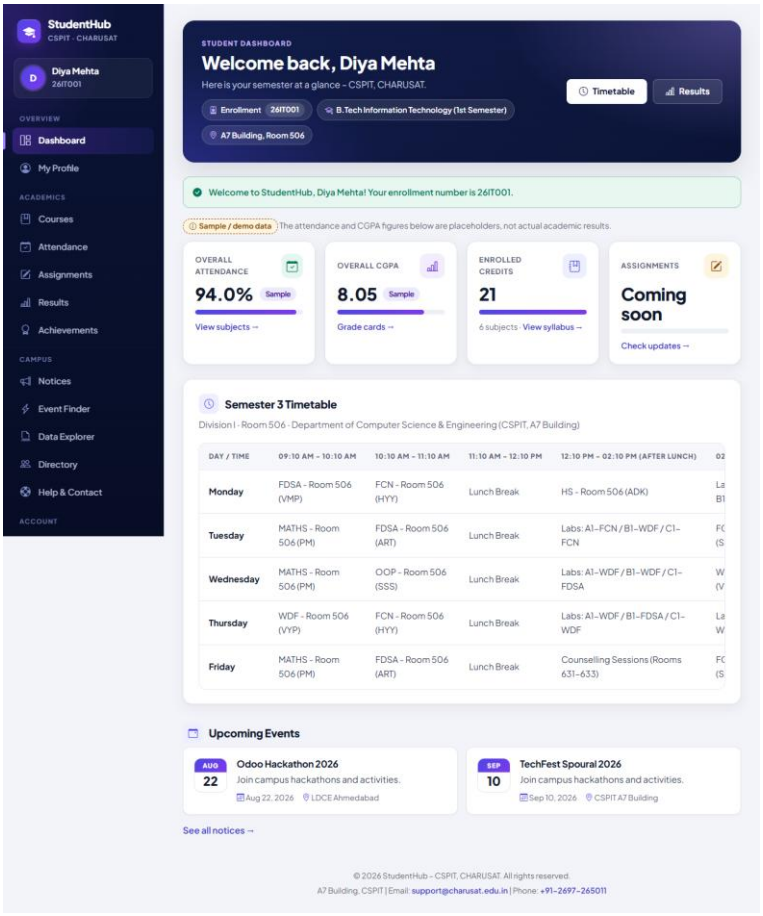


Figure 3: After a successful registration: Diya Mehta is signed in at once and redirected to dashboard.php with "Welcome to StudentHub, Diya Mehta! Your enrollment number is 26IT001."; sidebar and header show the linked record (26IT001, B.Tech Information Technology, 1st semester). Attendance and CGPA tiles are labelled sample data (registered-dashboard.png)

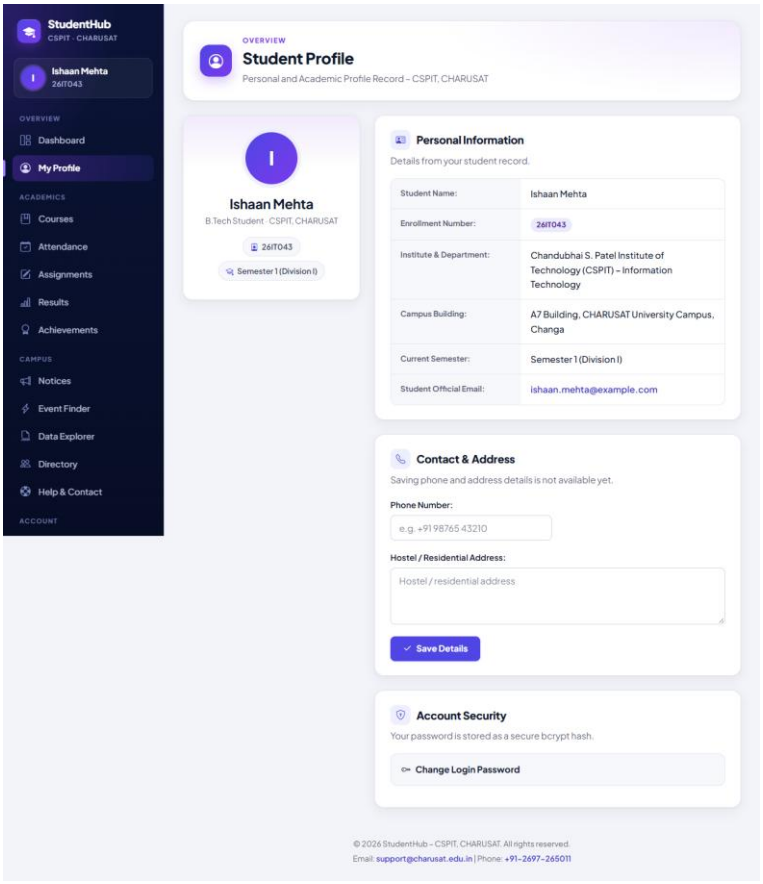


Figure 4: My Profile of Ishaan Mehta, read from the student record linked through students.user_id: enrollment number 26IT043, Information Technology, semester 1 (derived from admission year 2026), e-mail ishaan.mehta@example.com (03-student-profile.png)

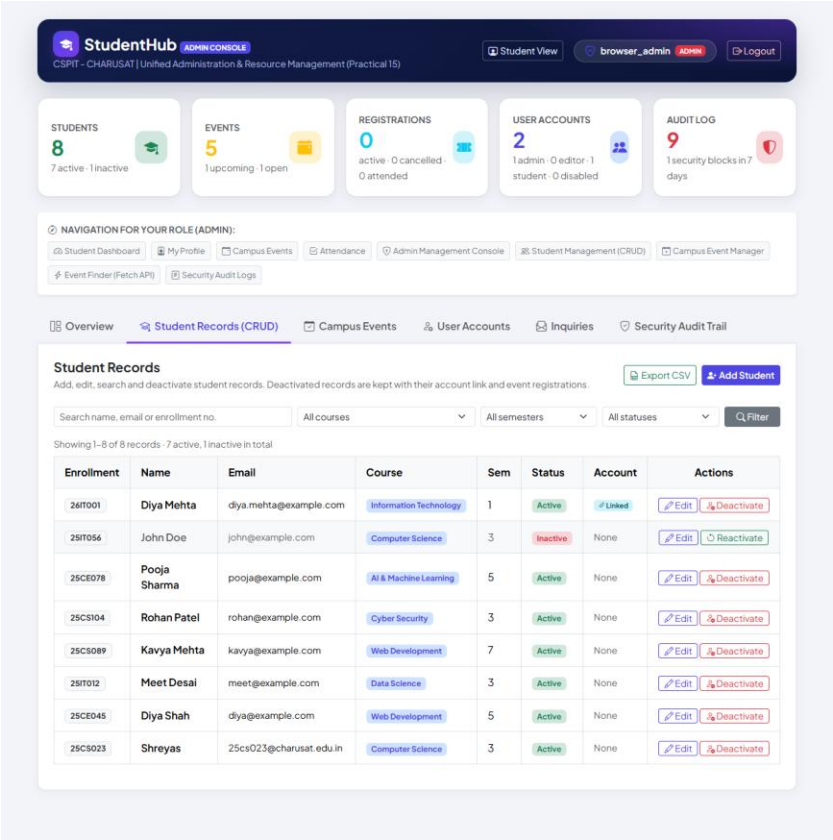


Figure 5: Admin Console → Student Records in the same run as Figures 1 and 3: the self-registered Diya Mehta (26IT001, Information Technology, semester 1) is Active with the account column "Linked"; the demo records from studenthub_demo.sql have no account ("None") (admin-console.png)

13. Testing and Results

All tests were executed on 31 October 2026 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and PHP's built-in server on Windows 11, against the disposable database `studenthub_auth_test` (built from `database/studenthub_demo.sql`, dropped afterwards). Test passwords are random and never printed; the live `studenthub_db` users and password hashes were compared before and after and were unchanged (TEST_REPORT §6).

Suite	Result	Output
`tests/test_auth.py` – HTTP tests of registration, login, sessions and RBAC	**132 / 132 passed**	`tests/test_auth_output.txt`
`tests/browser_auth.py` – Microsoft Edge (Figures 1, 3, 5)	**14 / 14 passed**	`tests/browser_auth_output.txt`
`tests/browser_e2e.py` – end-to-end scenario (Figures 2, 4)	**17 / 17 passed**	`tests/browser_e2e_output.txt`
`tests/run_all.py` – complete run of every suite	**841 passed, 0 failed**	`tests/run_all_output.txt`

The registration-related checks of `tests/test_auth.py` (the other checks belong to Practical 10):

Area	Checks	Examples (from test_auth_output.txt)
Valid registration and linkage	12	CSRF token and no role field in the form; 303 to dashboard.php; account and record created together with `students.user_id = users.id`; role `student` although the form sent `role=admin`; numbers 26CS001 and the next one; semester 1 for 2026, 25DS… and semester 3 for 2025; bcrypt hash accepted by `password_verify()`; signed in with welcome message; own profile; USER_REGISTER

Invalid input	12	invalid e-mail; password too short, without uppercase/number/special, longer than 72 bytes; passwords do not match; username starting with a digit or too short; name with digits/HTML; course, admission year or gender not in the list; terms not accepted
Duplicates and existing records	6	duplicate e-mail (other case, spaces) and username (other case) refused; no account created by refused forms; input escaped on redisplay; e-mail of a pre-existing student record refused and that record not linked or changed
CSRF / rollback / concurrency	1 + 2 + 1	no valid token → refused, nothing stored; forced failure → general message, both rows rolled back; 5 simultaneous registrations → 5 different, consecutive enrollment numbers
Static checks / server log	7 + 3	php -l on all 40 PHP files of studenthub/ and tools/; registration inserts the role as the literal 'student'; no PHP warnings; no password in the server log

Browser checks (Edge): `browser_auth.py` confirmed that the weak password is stopped in the browser and the values stay in the fields, that registration leads to the dashboard with the welcome message and enrollment number, that the sidebar shows the student's own name and number, and that the database contains the account and its linked record. `browser_e2e.py` step 1: "registration: account + linked student record with the next enrollment number, signed in". Both outputs report one known 404 for `/favicon.ico` (the portal has no favicon). The suite rows of the complete run are "Practicals 9/10 – registration, login, sessions, RBAC" 132 / 0, "Edge – registration, login, RBAC" 14 / 0 and "Edge – end-to-end (Practical 15)" 17 / 0.

14. Problems Found and Fixed

- **Accounts without a student record** showed a fixed name and roll number (Shreyas / 25CS023) on several pages (TEST_REPORT §2). Registration now creates the linked record, and an account without one sees only its own username and no enrollment number (TEST_REPORT §6; test "an account without a linked student record sees its own username …").
- **Old shared database:** the earlier Practical-9 code used the shared `user_registration` database. The portal uses only `studenthub_db`; `db.php` has no fallback, and the tests confirmed that an unavailable database gives HTTP 503 on the home, sign-in and dashboard pages instead of a login form or data from another database.
- **Time zone:** PHP used XAMPP's `Europe/Berlin` while MariaDB uses India time, so between midnight and about 04:30 the academic year used for enrollment numbers and dates could be one day behind; `db.php` now sets `Asia/Kolkata` (TEST_REPORT §7).
- **Browser test after Practical 5:** once the real-time checks were added, the weak-password check in `browser_auth.py` was changed to expect the browser to stop the form; the server refusal is tested over HTTP (TEST_REPORT §12).
- **Taking over existing records:** linking a registration to an existing record with the same e-mail could hand someone else's record to a new account; such registrations are refused and the administrator is asked to help.

15. Differences from the Original Practical and Limitations

Aspect	Original report (Practical-9/)	Integrated portal
Database	Own `studenthub_prac9`: `users` + `audit_logs`	Shared `studenthub_db`: `users` + linked `students` record
Fields	Full name, username, e-mail, password, confirmation, terms	+ gender, course, admission year (enrollment number, semester)

One transaction	User row + audit row	Account + student record; USER_REGISTER logged after the commit
After success	Confirmation page (Post/Redirect/Get)	Signed in at once; 303 to the dashboard with the enrollment number
CSRF failure / hash CHECK	HTTP 403; CHECK constraint on the hash format	Message in the form (HTTP 422); no CHECK – only the application writes hashes
Tests	93 / 93 (PHP) and 78 / 78 (Python)	132 / 132 HTTP, 14 / 14 and 17 / 17 Edge; complete run 841 / 0

- No e-mail verification or password reset (the portal sends no e-mail), no rate limiting and no CAPTCHA – the canvas CAPTCHA of Practical 5 was not carried into the portal (TEST_REPORT §12–§13).
- The duplicate messages reveal that a username or e-mail address is already registered (required by the practical).
- Automatic enrollment numbers exist only for the four registration courses, at most 999 per prefix. For a self-registered student the e-mail is stored in `users` and in `students`.
- An all-empty POST is not among the saved checks; empty fields are handled by the "required" branches shown in Section 7. Tested only with PHP's built-in server, MariaDB 10.4.32 and Microsoft Edge.

16. Conclusion

All seven evaluation requirements are met in the integrated portal. Every field is validated on the server (with browser checks only for convenience), duplicate e-mail addresses and usernames are refused in PHP and by UNIQUE keys, passwords are stored only as bcrypt hashes, and every query with user input is a MySQLi prepared statement. One transaction creates the account and its linked student record with the next enrollment number, so a new student can use the dashboard, the profile and event registration immediately, and a failure leaves nothing behind. Both extensions are implemented: a CSRF token on the form and an audit entry for every registration. 132 HTTP checks, 14 + 17 browser checks and the complete run of 841 checks passed.

17. References

- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – `studenthub/register.php`, `studenthub/includes/registration.php`, `studenthub/includes/auth.php`, `tests/`, `evidence/practical-09-10-integration/`, `TEST_REPORT.md` §6; commit `8011d39`
- PHP Manual – password_hash: https://www.php.net/manual/en/function.password-hash.php
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – random_bytes and hash_equals: https://www.php.net/manual/en/function.random-bytes.php, https://www.php.net/manual/en/function.hash-equals.php
- PHP Manual – Validate filters (FILTER_VALIDATE_EMAIL): https://www.php.net/manual/en/filter.filters.validate.php
- OWASP – Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html